Actividad final integradora — La API REST de Palco

Desarrollo de Aplicaciones II · UADE · Clase 10 (Unidad VI, Servicios REST) · Dominio: venta de entradas con reventa oficial

Integrantes: Santiago Loto, Sebastian Deya, Julian Zuchowicki, Valentina Frisoli, Joaquin Molina.

1. Catálogo de endpoints

Base: /api/v1. Recursos en plural y en sustantivos; los verbos los pone HTTP. Todos devuelven application/json y los errores application/problem+json (RFC 9457). Madurez Richardson nivel 2, con nivel 3 (_links) en el recurso Venta.

#	Método y URI	Request	Éxito	Errores (code)
1	GET /eventos?categoria=&q=&pagina=&tam=	Query; público	200 lista paginada	400 parámetro inválido
2	GET /eventos/{slug}	Header If-None-Match; público	200 + ETag · 304 sin cambios	404 evento inexistente
3	POST /eventos	JSON evento + sectores; rol organizer	201 + Location	400 · 401 · 403 · 422 fecha pasada o sector sin precio
4	POST /ventas	Idempotency-Key; {eventoId, items[], titulares[]}; rol user	201 + Location; reserva 10 min	400 · 401 · 403 identidad no verificada · 409 sin cupo · 422 más de 6 entradas o DNI repetido
5	POST /ventas/{id}/pagos	Idempotency-Key; {medioDePago, token}	201 (venta CONFIRMADA); 202 si la pasarela deja el pago pendiente	402 rechazado · 404 · 409 reserva vencida o ya paga · 504 pasarela sin respuesta
6	GET /entradas/{id}	Bearer; solo el titular	200 + ETag (estado, QR, _links)	401 · 404 (no revela existencia ajena)
7	POST /reventas	{entradaId, precio}; titular verificado	201 + Location	403 no titular · 409 ya publicada · 422 precio sobre tope (+10%), bajo 50% o faltan menos de 3 h
8	DELETE /reventas/{id}	Bearer; dueño de la publicación	204 sin cuerpo (idempotente)	403 · 404 · 409 ya vendida
9	POST /pagos/webhook	Evento firmado de Mercado Pago	200 vacío (acuse inmediato)	401 firma inválida · 400
10	POST /validacion/escaneos	{codigoQr}; rol staff	200 {resultado: VALIDA}	404 QR desconocido · 409 ya usada · 422 evento equivocado

Notas: PUT/PATCH sobre /eventos/{slug} queda para el organizador (PATCH, con If-Match para evitar pisar cambios). El cupo lo modifica solo el rol servicio vía POST /eventos/{id}/sectores/{sid}/cupo (llamada entre servicios, nunca el cliente).

2. Contrato OpenAPI (fragmento)

Se escribió contract-first. Acá van los endpoints 2 (GET detalle) y 4 (POST ventas, con respuestas de error). El archivo completo, con 2 endpoints más (pagos y reventas) y los schemas, se entrega como openapi-palco-v1.yaml.

```
openapi: 3.1.0
info:
  title: Palco API
  version: 1.0.0
  description: >
    API REST de Palco, plataforma de venta de entradas con reventa oficial (tope +10%).
    Montos en pesos enteros. Fechas ISO 8601 con zona horaria. Errores en formato Problem Details (RFC 9457).
servers:
  - url: https://api.palco.example/api/v1
    description: Base versionada en la URI
security:
  - bearerAuth: []
tags:
  - name: eventos
  - name: ventas
  - name: reventas

paths:
  /eventos/{slug}:
```

```
  get:
    tags: [eventos]
    summary: Detalle de un evento con sus sectores y cupo disponible
    security: []                    # catálogo público
    parameters:
      - name: slug
        in: path
        required: true
        schema: { type: string, example: atletico-norte-racing-sur }
      - name: If-None-Match
        in: header
        required: false
        schema: { type: string }
        description: ETag de la última copia; si no cambió responde 304 sin cuerpo.
    responses:
      '200':
        description: Evento encontrado
        headers:
          ETag:
            schema: { type: string }
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Evento' }
      '304':
        description: Sin cambios, usar la copia local
      '404':
        description: Evento inexistente
        content:
          application/problem+json:
            schema: { $ref: '#/components/schemas/Problema' }

/ventas:
  post:
    tags: [ventas]
    summary: Iniciar una compra y reservar el cupo durante 10 minutos
    description: Requiere rol `user` con identidad verificada. Es idempotente por `Idempotency-Key`.
    parameters:
      - $ref: '#/components/parameters/IdempotencyKey'
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: '#/components/schemas/NuevaVenta' }
    responses:
      '201':
        description: Compra iniciada, cupo reservado
        headers:
          Location:
            schema: { type: string, example: /api/v1/ventas/V-00231 }
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Venta' }
      '400':
        description: Cuerpo mal formado o campos inválidos
        content:
          application/problem+json:
            schema: { $ref: '#/components/schemas/Problema' }
      '401': { $ref: '#/components/responses/NoAutenticado' }
      '403': { $ref: '#/components/responses/SinPermiso' }
      '409':
        description: El sector no tiene cupo suficiente
        content:
          application/problem+json:
            schema: { $ref: '#/components/schemas/Problema' }
      '422':
        description: Viola una regla de negocio (más de 6 entradas, DNI repetido)
        content:
          application/problem+json:
            schema: { $ref: '#/components/schemas/Problema' }
```

3. Ejemplos JSON

Request exitoso — POST /api/v1/ventas

```
POST /api/v1/ventas
Authorization: Bearer eyJhbGciOiJSUzI1NiJ9...
Idempotency-Key: 6b1f1c5e-0d6e-4a0f-9c6e-6f1e2a7d9a10
Content-Type: application/json

{
  "eventoId": "ev-1",
  "items": [ { "sectorId": "sec-1-1", "cantidad": 2 } ],
  "titulares": [
    { "nombre": "Ana Gómez",  "dni": "30123456" },
    { "nombre": "Luis Pérez", "dni": "31234567" }
  ]
}
```

Response exitoso

```
HTTP/1.1 201 Created
Location: /api/v1/ventas/V-00231
Content-Type: application/json

{
  "id": "V-00231",
  "estado": "EN_CHECKOUT",
  "facturaEstado": "NO_APLICA",
  "montoTotal": 60500,
  "venceReserva": "2026-10-19T21:10:00-03:00",
  "_links": {
    "self":   { "href": "/api/v1/ventas/V-00231", "method": "GET" },
    "pagar":  { "href": "/api/v1/ventas/V-00231/pagos", "method": "POST" },
    "cancelar": { "href": "/api/v1/ventas/V-00231", "method": "DELETE" }
  }
}
```

Error — mismo POST cuando el sector no tiene cupo (Problem Details, RFC 9457)

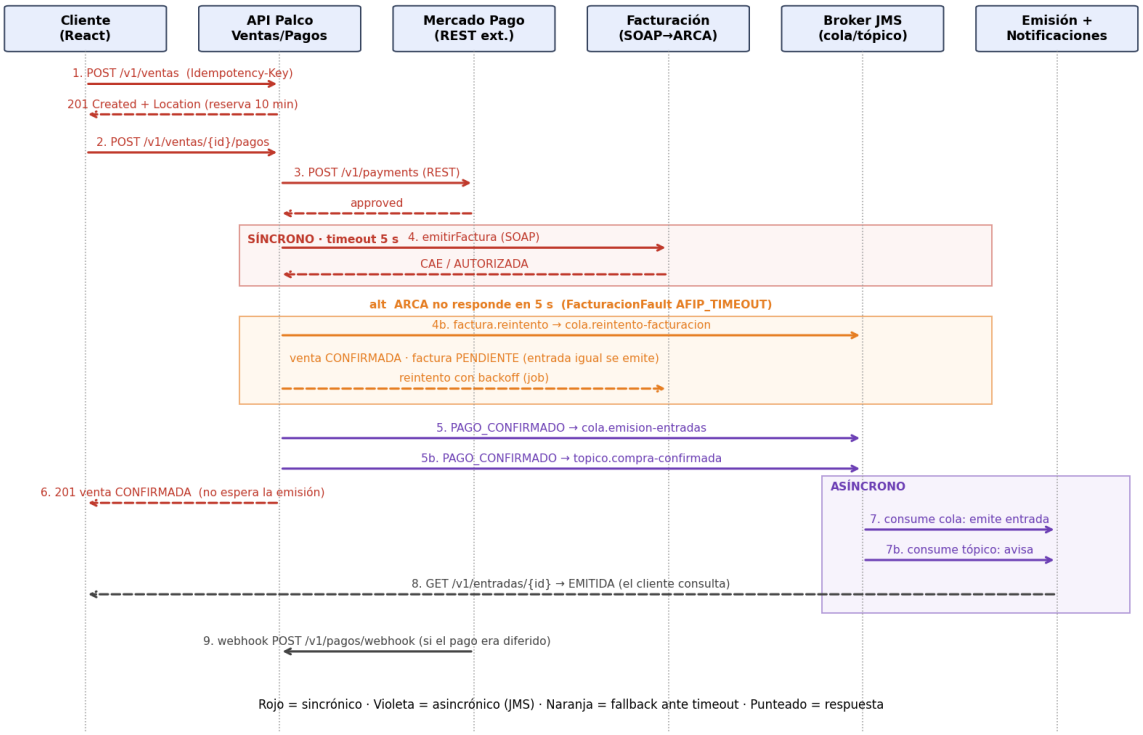
```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "/problemas/cupo-agotado",
  "title": "No hay cupo suficiente en el sector",
  "status": 409,
  "detail": "Pediste 2 entradas de Platea A y quedan 1.",
  "instance": "/api/v1/ventas",
  "sectorId": "sec-1-1",
  "disponibles": 1
}
```

Estado actual del repo: el servicio de Eventos responde errores con {estado, mensaje, campos} (ErrorRespuesta). Falta migrarlo a Problem Details con un ExceptionMapper común en backend/common; está planificado en la tarea de ventas.

4. Diagrama de secuencia de punta a punta

Confirmar una compra integra las Clases 8, 9 y 10: API REST propia → Mercado Pago (REST externo) → facturación SOAP legado (ARCA, timeout 5 s) → eventos JMS (cola y tópico).



- **Sincrónico:** pasos 1–4: el cliente espera. Hace falta saber si el cobro y la factura salieron antes de confirmar.
- **Asincrónico:** pasos 5–7: emitir la entrada (cola, un solo consumidor) y avisar (tópico, varios suscriptores). El cliente no espera; consulta GET /entradas/{id}.

5. Decisiones justificadas

- **Idempotencia.** POST /ventas y /pagos exigen Idempotency-Key (UUID). El servidor guarda clave → respuesta; si la red falla y el cliente reintenta, recibe la misma respuesta y no se cobra dos veces. GET, PUT y DELETE ya son idempotentes por semántica. El webhook se deduplica por el id del pago de Mercado Pago, porque la pasarela reenvía.
- **Versionado.** En la URI (/api/v1). Es visible, cacheable y simple de probar en Postman. Solo cambia la versión mayor ante un cambio incompatible (quitar o renombrar campos); agregar campos opcionales no rompe y no la cambia. Mantenemos la v1 vigente mientras haya clientes.
- **Seguridad.** JWT RS256 en Authorization: Bearer (MicroProfile JWT; las claves públicas están en /jwks de Usuarios). Autorización declarativa con @RolesAllowed (user, organizer, staff, servicio). Solo HTTPS. Los DNI viajan en el cuerpo, nunca en la URL. El webhook se valida por firma, no por JWT. Un recurso ajeno devuelve 404 y no 403, para no revelar que existe. El 429 limita el login y la compra para frenar bots.
- **Contrato.** Se usan DTO, nunca entidades JPA, para no filtrar el modelo de base. Precios en pesos enteros (long). Paginación por página en el catálogo (acotado) y por cursor si crece.
- **Pregunta clave: ¿qué responde la API si el SOAP legado no contesta en 5 s?** La API no espera más de 5 s a ARCA. Si vence el timeout o llega FacturacionFault (AFIP_TIMEOUT o AFIP_NO_DISPONIBLE), el pago ya está cobrado, así que no puede fallar la compra. Responde 201 con la venta en estado CONFIRMADA y facturaEstado PENDIENTE, publica factura.reintento en cola.reintento-facturacion y un job reintenta con backoff exponencial. La entrada se emite igual. Cuando llega el CAE, la factura pasa a AUTORIZADA y se avisa por el tópico. No devolvemos 504 porque eso haría creer al cliente que no pagó y reintentaría el pago (el Idempotency-Key lo protege igual, pero la experiencia sería mala). 504 queda reservado para cuando falla Mercado Pago, donde sí no hay cobro. Si tras N reintentos sigue fallando, se alerta al organizador.
- **Por qué 201 y no 202.** Se prefiere 201 a 202 porque el recurso Venta sí se creó y quedó consistente; 202 se usaría si el cobro mismo fuera asincrónico (pago diferido con webhook).

Bonus (Entrega 4)

GET /api/eventos y GET /api/eventos/{slug} ya están implementados con Jakarta REST (JAX-RS) en el servicio de Eventos, con @PermitAll y el POST con @RolesAllowed("organizer"). Falta subir el prefijo /v1 y probarlos en Postman.